

ΕΝΟΤΗΤΑ 4 ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ

4.1 Εισαγωγή

Η Τεχνητή Νοημοσύνη (ΤΝ) είναι ένας κλάδος της Πληροφορικής που στοχεύει στη δημιουργία μηχανών ικανών να εκτελούν εργασίες που συνήθως απαιτούν ανθρώπινη νοημοσύνη. Τα τελευταία χρόνια έχουμε γίνει μάρτυρες μιας επανάστασης στον τομέα της Τεχνητής Νοημοσύνης, η οποία προέρχεται από τα μεγάλα γλωσσικά μοντέλα που έχουν αναπτυχθεί, όπως είναι το ChatGPT της OpenAI και το Gemini της Google.

Τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) είναι μια κατηγορία νευρωνικών δικτύων που έχουν εκπαιδευτεί σε τεράστιες ποσότητες κειμένου. Αυτά τα μοντέλα είναι σε θέση να εκτελούν ένα ευρύ φάσμα εργασιών που σχετίζονται με τη γλώσσα. Τα μοντέλα αυτά προβλέπουν την επόμενη λέξη σε μια ακολουθία λέξεων με βάση τις προηγούμενες. Για να υπολογίσει αυτήν την πιθανότητα με ακρίβεια, το μοντέλο προ-εκπαιδευτεί σε πάρα πολλά κείμενα, στην περίπτωση του ChatGPT σε όλα τα κείμενα που βρίσκονται διαθέσιμα στον Παγκόσμιο Ιστό και όχι μόνο. Από εκεί προκύπτει το ακρωνύμιο GPT που σημαίνει Generative Pretrained Transformer (δηλαδή, Παραγωγικός Προ-εκπαιδευμένος Μετασχηματιστής). Η λέξη μετασχηματιστής αναφέρεται στο είδος των νευρωνικών δικτύων που χρησιμοποιείται. Στην ενότητα αυτή θα μάθετε τι είναι ένα νευρωνικό δίκτυο και θα υλοποιήσετε μια εφαρμογή Τεχνητής Νοημοσύνης που μπορεί να εκτελεστεί στο κινητό σας τηλέφωνο.



Τι είναι ένα νευρωνικό δίκτυο;



Ένα νευρωνικό δίκτυο είναι ένα υπολογιστικό μοντέλο που εμπνέεται από τη βιολογική δομή και λειτουργία του εγκεφάλου. Αποτελείται από ένα σύνολο συνδεδεμένων κόμβων ή "νευρώνων" που δουλεύουν μαζί για να επεξεργαστούν πληροφορίες και να λύσουν προβλήματα. Οι νευρώνες αυτοί οργανώνονται σε επίπεδα (layers):

1. **Εισόδου (Input Layer):** Λαμβάνει τα αρχικά δεδομένα.
2. **Κρυφά Επίπεδα (Hidden Layers):** Μεσολαβούν μεταξύ της εισόδου και εξόδου, εκτελώντας υπολογισμούς και μετασχηματισμούς. Ο αριθμός και το μέγεθος των κρυφών επιπέδων μπορεί να διαφέρει.
3. **Εξόδου (Output Layer):** Παράγει το τελικό αποτέλεσμα ή πρόβλεψη.

Οι συνδέσεις μεταξύ των νευρώνων έχουν βάρη (weights) που προσαρμόζονται κατά τη διάρκεια της εκπαίδευσης του μοντέλου για να ελαχιστοποιήσουν το σφάλμα της πρόβλεψης. Η διαδικασία εκπαίδευσης περιλαμβάνει τη χρήση αλγορίθμων βελτιστοποίησης, όπως ο αλγόριθμος της αντίστροφης διάδοσης (backpropagation), για την προσαρμογή αυτών των βαρών.

Τα νευρωνικά δίκτυα χρησιμοποιούνται σε διάφορους τομείς, όπως η αναγνώριση εικόνας, η επεξεργασία φυσικής γλώσσας, η πρόβλεψη δεδομένων και πολλά άλλα, λόγω της ικανότητάς τους να μαθαίνουν και να γενικεύουν από τα δεδομένα.

Εικόνα 4.1 Ρωτήσαμε το ChatGPT τι είναι ένα νευρωνικό δίκτυο

4.2 Μηχανική Μάθηση με Νευρωνικά Δίκτυα

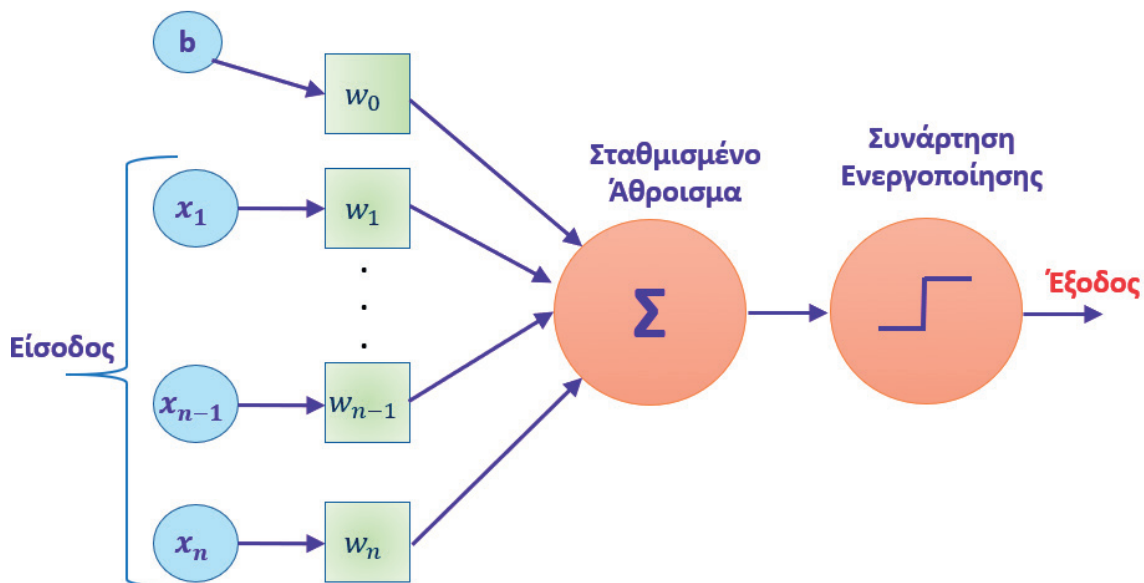
Το **perceptron** θεωρείται το βασικό δομικό στοιχείο ενός τεχνητού νευρωνικού δικτύου (artificial neural network). Θα μπορούσε να θεωρηθεί ως το μαθηματικό/υπολογιστικό μοντέλο ενός βιολογικού νευρώνα. Επινόηθηκε το 1957 από τον Frank Rosenblatt. Αποτελείται από έναν μόνο νευρώνα, ο οποίος λαμβάνει πολλαπλές εισόδους και παράγει μια έξοδο. Κάθε είσοδος έχει ένα βάρος, το οποίο καθορίζει τη σημασία της για την τελική έξοδο. Η μεροληπτική σταθερά b (bias) δε θεωρείται είσοδος, μπορεί όμως να αλλάξει ανάλογα με το πεδίο τιμών στο οποίο θέλουμε να βρίσκεται το αποτέλεσμα.

Τα βασικά βήματα της λειτουργίας του perceptron είναι τα εξής:

- Υπολογισμός του σταθμισμένου αθροίσματος:** Το perceptron υπολογίζει το σταθμισμένο άθροισμα των εισόδων του, πολλαπλασιάζοντας κάθε είσοδο με το αντίστοιχο βάρος της και προσθέτοντας τα αποτελέσματα. Για παράδειγμα αν έχουμε εισόδους

$$x_1 = 1, x_2 = 0, x_3 = 0.5 \text{ και } \text{βάρη } w_1 = 0.2, w_2 = 0.4, w_3 = 0.6, \text{ το } \text{άθροισμα είναι} \\ b + x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 = -2 + 0.2 + 0 + 0.3 = -1.5 < 0$$

- Εφαρμογή της συνάρτησης ενεργοποίησης:** Στη συνέχεια, το perceptron εφαρμόζει μια συνάρτηση ενεργοποίησης στο άθροισμα. Η πιο κοινή συνάρτηση ενεργοποίησης είναι η βηματική συνάρτηση (step function), η οποία επιστρέφει 1 εάν το άθροισμα είναι μεγαλύτερο ή ίσο με 0 αλλιώς επιστρέφει 0. Αφού το αποτέλεσμα είναι $-1.5 < 0$ τότε η συνάρτηση επιστρέφει 0.
- Έξοδος:** Η έξοδος του perceptron είναι η τιμή που επιστρέφει η συνάρτηση ενεργοποίησης, άρα 0.



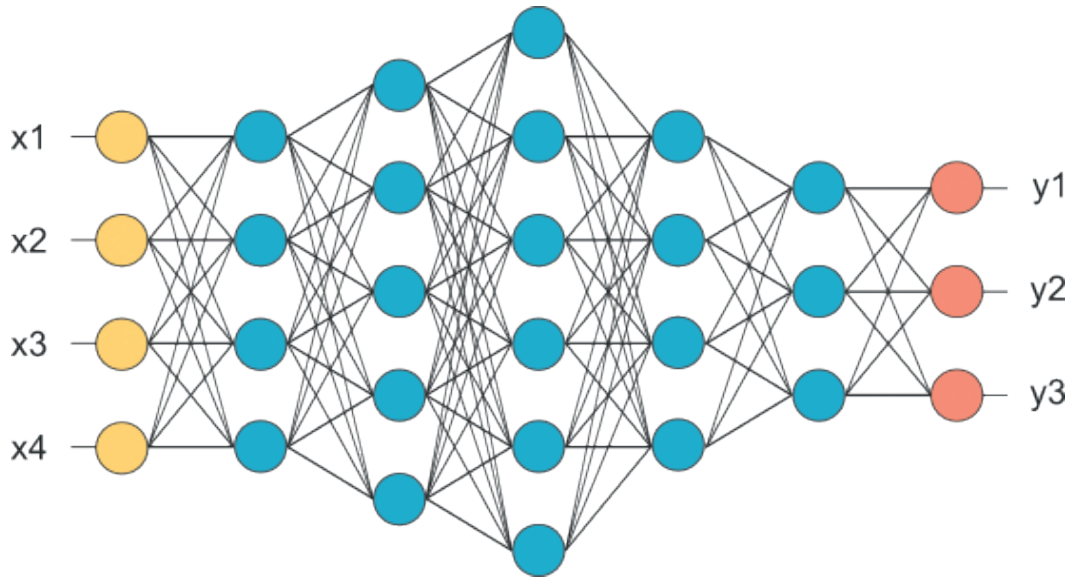
Εικόνα 4.2 Perceptron: Το δομικό στοιχείο ενός νευρωνικού δικτύου

Η εκπαίδευση ενός perceptron περιλαμβάνει την προσαρμογή των βαρών με βάση το σύνολο δεδομένων εκπαίδευσης (training data set). Συγκρίνεται η υπολογισμένη έξοδος με την επιθυμητή (πραγματική) έξοδο. Τα βάρη και η μεροληπτική σταθερά b (bias) προσαρμόζονται με βάση το σφάλμα της πρόβλεψης. Για παράδειγμα αν η έξοδος που περιμέναμε ήταν 1, τότε αν αλλάξουμε τη μεροληπτική σταθερά από -2 σε 0 , το αποτέλεσμα θα γίνει $0.5 > 0$ και η συνάρτηση ενεργοποίησης θα επιστρέψει έξοδο 1, όπως αναμένουμε. Θα μπορούσαμε επίσης να τροποποιήσουμε τα βάρη με τιμές $w_1 = 2, w_3 = 1$, οπότε το σταθμισμένο άθροισμα γίνεται $-2 + 2 + 0 + 0.5 = 0.5 > 0$.

Φυσικά η τροποποίηση του b ή των βαρών δε γίνεται τόσο απλά, αλλά με συγκεκριμένους αλγορίθμους.

Ένας από αυτούς είναι γνωστός ως **gradient descent** και η λειτουργία του βασίζεται σε ανώτερα μαθηματικά. Ο στόχος του αλγορίθμου είναι να ελαχιστοποιήσει τη διαφορά των εξόδων με τα επιθυμητά αποτελέσματα, προσαρμόζοντας τα βάρη του μοντέλου μέσω επαναλαμβανόμενων βημάτων, προς την κατεύθυνση της μεγαλύτερης μείωσης αυτής της διαφοράς.

Ένα νευρωνικό δίκτυο αποτελείται από πάρα πολλά perceptrons τα οποία κατανέμονται σε επίπεδα. Για παράδειγμα, το ChatGPT 4.0 έχει περίπου 100 δισεκατομμύρια perceptrons που κατανέμονται σε 100 επίπεδα.



Εικόνα 4.3 Νευρωνικό δίκτυο με πολλά επίπεδα

Τα νευρωνικά δίκτυα εκπαιδεύονται ως εξής: Το δίκτυο λαμβάνει ένα σύνολο δεδομένων εκπαίδευσης, το οποίο περιλαμβάνει ζεύγη εισόδου-εξόδου. Το δίκτυο ρυθμίζει τα βάρη των συνδέσεων του με στόχο να παράγει τις σωστές εξόδους για δεδομένες εισόδους με την εφαρμογή αλγορίθμων, όπως ο gradient descent. Μετά την εκπαίδευση, τα νευρωνικά δίκτυα μπορούν να χρησιμοποιηθούν, για να προβλέψουν εξόδους για νέα άγνωστα δεδομένα.

Τα νευρωνικά δίκτυα μπορούν να χρησιμοποιηθούν για:

- ▶ αναγνώριση αντικειμένων σε εικόνες, όπως πρόσωπα, αυτοκίνητα και χειρόγραφα
- ▶ ανάλυση ιατρικών εικόνων (π.χ. ακτινογραφίες, αξονικές τομογραφίες) για διάγνωση ασθενειών
- ▶ επεξεργασία φυσικής γλώσσας μέσα από την ταξινόμηση λέξεων και φράσεων
- ▶ λήψη αποφάσεων σε περίπλοκα προβλήματα, όπως η διάγνωση ασθενειών και η πρόβλεψη χρηματιστηριακών αγορών
- ▶ ρομποτική και αυτόνομη οδήγηση

Είναι σημαντικό να σημειωθεί ότι η χρήση νευρωνικών δικτύων εγείρει και ηθικά ζητήματα, όπως η προκατάληψη και η διαφάνεια. Είναι απαραίτητο να αναπτύξουμε υπεύνη και ηθική Τεχνητή Νοημοσύνη που να ωφελεί την κοινωνία στο σύνολό της.

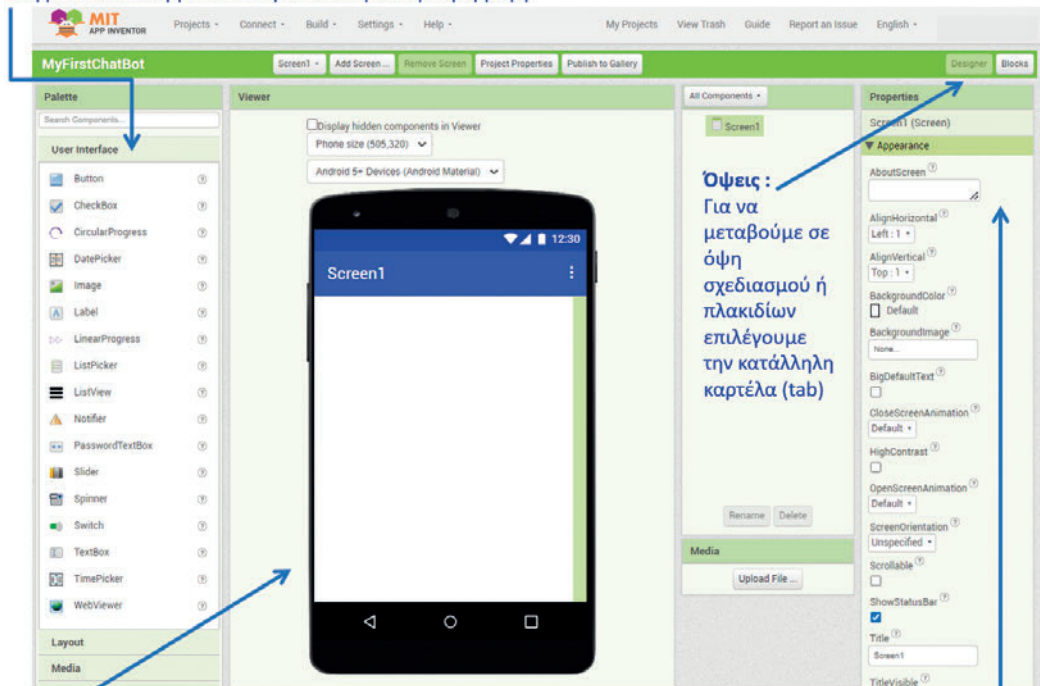
4.3 Το περιβάλλον App Inventor

Το App Inventor (ή MIT AI2) είναι ένα περιβάλλον οπτικού προγραμματισμού πλακιδίων (block programming) υψηλού επιπέδου, που αρχικά δημιουργήθηκε από την Google και τώρα υποστηρίζεται από το Ινστιτούτο Τεχνολογίας της Μασαχουσέτης (MIT). Επιτρέπει στους χρήστες να δημιουργούν εφαρμογές για συσκευές Android ή iOS με έναν εύκολο και διαδραστικό τρόπο, χρησιμοποιώντας μια γραφική διεπαφή που βασίζεται σε πλακίδια (blocks). Το App Inventor είναι δωρεάν και ανοικτού κώδικα. Χρειάζεται να εγκαταστήσουμε την εφαρμογή στον υπολογιστή και στο κινητό μας από τη διεύθυνση <http://appinventor.mit.edu/>.



Για να συνδεθεί το κινητό τηλέφωνο με την εφαρμογή στον υπολογιστή μας, πρέπει να είναι συνδεδεμένες και οι δυο συσκευές στο ίδιο ασύρματο δίκτυο. Στη συνέχεια επιλέγουμε από το μενού Connect → AI Companion και με σάρωση του QR-code γίνεται η σύνδεση των δυο συσκευών και μπορούμε να βλέπουμε στο κινητό μας άμεσα τις αλλαγές που κάνουμε στην εφαρμογή στο περιβάλλον προγραμματισμού.

Παλέτα: Επιλέγουμε τα αντικείμενα που θέλουμε και τα σέρνουμε στη μεσαία οθόνη για να τα προσθέσουμε στην εφαρμογή

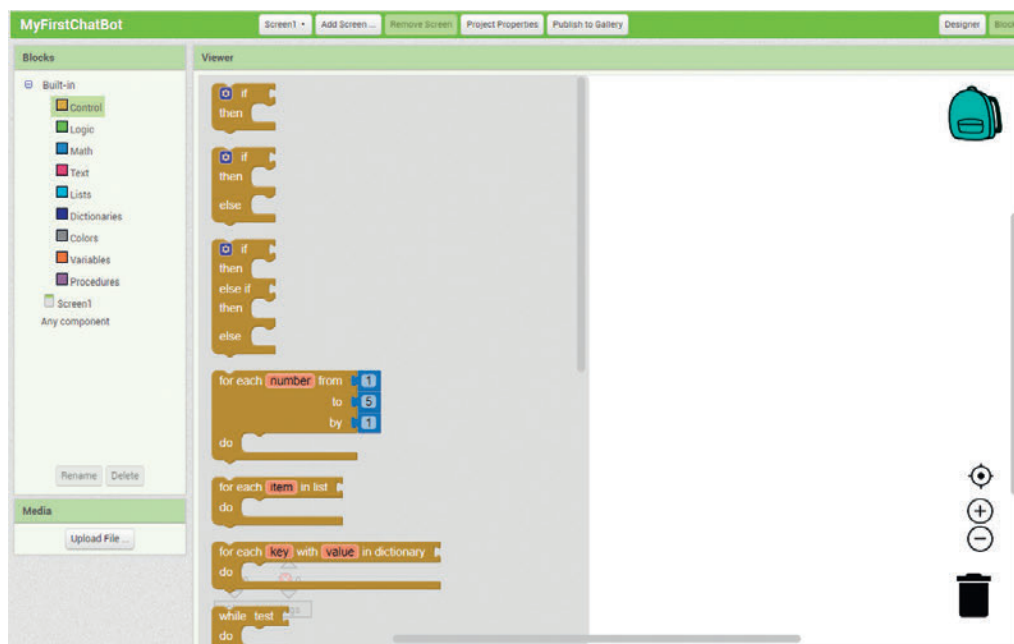


Όψη Σχεδιασμού: Στο κέντρο βλέπουμε την οθόνη του κινητού μαζί με τα αντικείμενα που έχουμε επιλέξει

Ιδιότητες: Δεξιά βλέπουμε τις ιδιότητες του αντικείμενου που έχουμε επιλέξει

Εικόνα 4.4 Το περιβάλλον προγραμματισμού App Inventor στην όψη σχεδιασμού (Designer)

Σε διαφορετική περίπτωση μπορούμε να συνδέσουμε τις δυο συσκευές με καλώδιο. Αν δε θέλουμε να χρησιμοποιήσουμε κινητό τηλέφωνο, το App Inventor διαθέτει έναν εξομοιωτή (emulator), τον οποίο μπορούμε να εγκαταστήσουμε στον υπολογιστή μας.

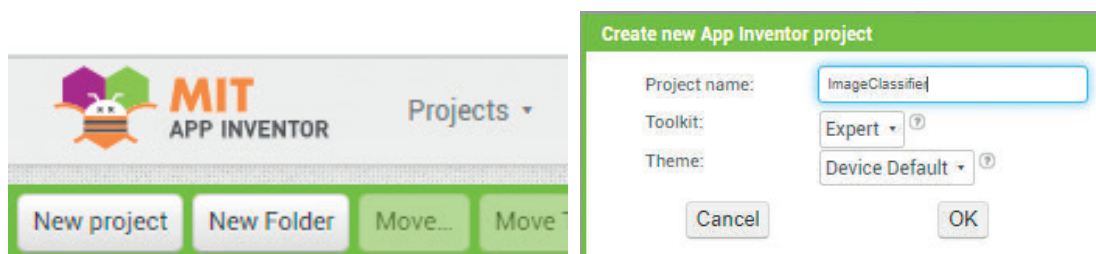


Εικόνα 4.5 Το περιβάλλον προγραμματισμού App Inventor στην όψη πλακιδίων (Blocks)

Η κλασική δομή του περιβάλλοντός του αποτελείται από: (α) τον Designer (Σχεδιαστή), όπου ο χρήστης επιλέγει τα συστατικά μέρη για την εφαρμογή που αναπτύσσει, και (β) τον Blocks Editor (Συντάκτη πλακιδίων), όπου ο χρήστης συνδυάζει οπτικά τα πλακίδια του προγράμματος, για να ορίσει τη συμπεριφορά των μερών της εφαρμογής (μοιάζει με τη συναρμολόγηση ενός πάζλ). Τα πλακίδια είναι ταξινομημένα σε διαφορετικά χρώματα, ανάλογα με τη λειτουργία που επιτελούν.

4.4 Αναγνώριση εικόνων με αξιοποίηση Τεχνητής Νοημοσύνης

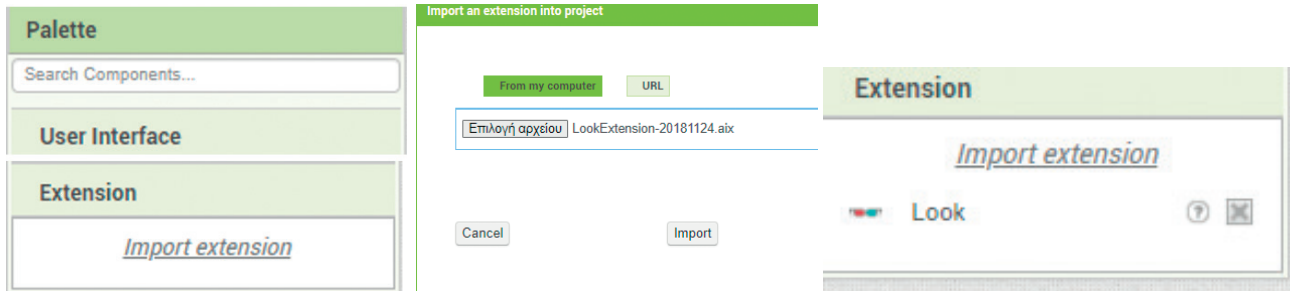
Θα αναπτύξουμε μια απλή εφαρμογή για κινητές συσκευές στο App Inventor η οποία θα αναγνωρίζει αντικείμενα σε εικόνες. Αρχικά πρέπει να δημιουργήσουμε ένα νέο έργο και να του δώσουμε ένα όνομα.



Για να μπορέσει η εφαρμογή μας να αναγνωρίζει εικόνες, θα χρησιμοποιήσουμε τη βιβλιοθήκη Look, η οποία παρέχει ένα έτοιμο νευρωνικό δίκτυο που έχει εκπαιδευτεί με διάφορες εικόνες. Για να προσαρτήσουμε τη βιβλιοθήκη στο πρόγραμμά μας και να χρησιμοποιήσουμε το νευρωνικό δίκτυο, θα πρέπει πρώτα να μεταβούμε στη διεύθυνση των επεκτάσεων του App Inventor, <https://mit-cml.github.io/extensions/> και από εκεί να κατεβάσουμε το αντίστοιχο αρχείο με κατάληξη aix.

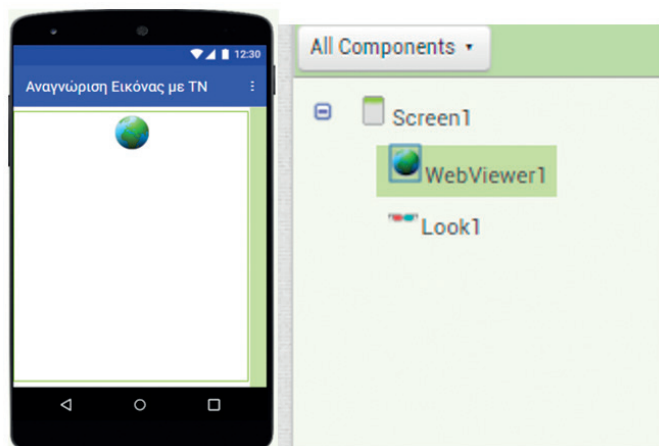
LookExtension	Adds object recognition using a neural network compiled into the extension.	MIT App Inventor	20181124	LookExtension.aix
---------------	---	------------------	----------	-----------------------------------

Στη συνέχεια από την παλέτα των εργαλείων στην τελευταία ομάδα εντολών extension (Επέκταση) εισάγουμε στο έργο μας την επέκταση LookExtension. Επιλέγουμε το αρχείο από τον φάκελο στον οποίο το έχουμε αποθηκεύσει και το προσαρτούμε (import) στην εφαρμογή μας. Τώρα έχει προστεθεί το Look στις επεκτάσεις.

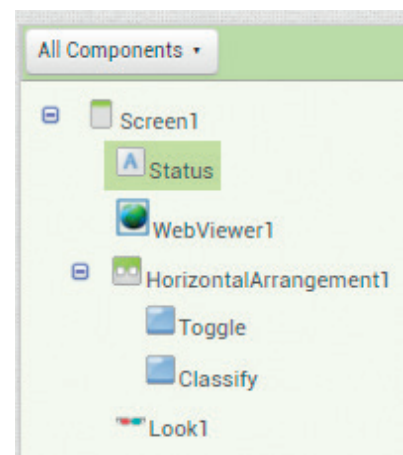


Εκτός από την επέκταση Look χρειάζεται να προσθέσουμε και ένα αντικείμενο WebView από την ομάδα User Interface. Το αντικείμενο αυτό συνδέεται με την κάμερα του κινητού μας και θα εμφανίζει στην οθόνη την εικόνα που βλέπουμε από την κάμερα του κινητού μας. Αν υπάρχει ένα αντικείμενο στην εικόνα αυτή, η εφαρμογή μας θα προσπαθήσει να «μαντέψει» τι είναι.

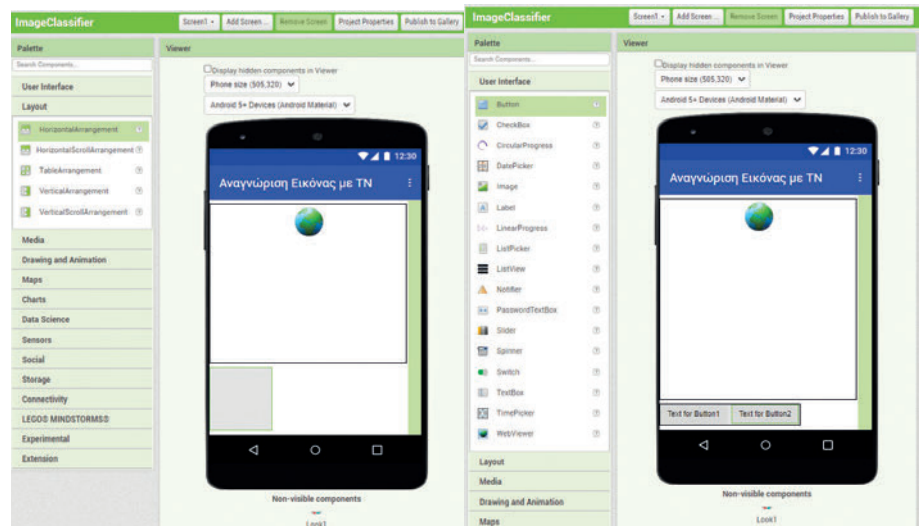
Τα αντικείμενα που θα χρειαστούμε και θα προσθέσουμε στο έργο μας είναι τα παρακάτω:



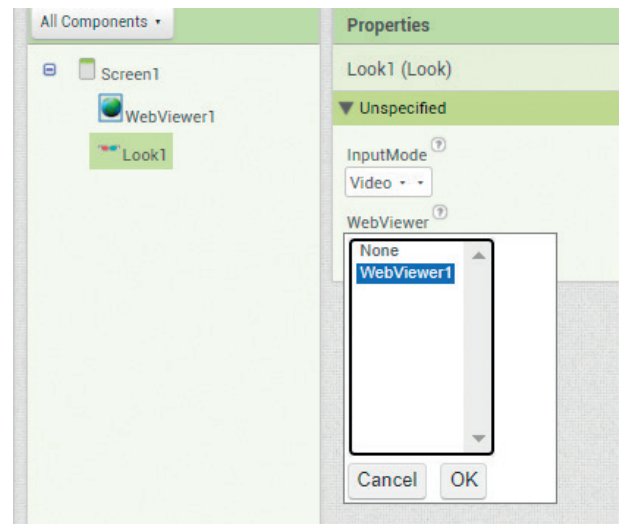
- ▶ μια ετικέτα κειμένου (Label), στην οποία θα γράφουμε τα αποτελέσματα της αναγνώρισης εικόνας και την οποία ονομάσαμε Status.
- ▶ δυο κουμπιά (Button), ένα για την εναλλαγή κάμερας και ένα για την αναγνώριση εικόνας, που τα ονομάσαμε Toggle και Classify.
- ▶ Ένα αντικείμενο HorizontalArrangement από την ομάδα Layout. Το αντικείμενο θα περιέχει τα δυο κουμπιά, τα οποία θα τοποθετήσει σε οριζόντια στοίχιση.
- ▶ Ένα αντικείμενο WebView, το οποίο θα εμφανίζει στην κεντρική οθόνη την εικόνα από την κάμερα το κινητού μας.
- ▶ Το αντικείμενο/επέκταση Look, που θα κάνει όλη τη δουλειά.



Αρχικά επιλέγουμε το αντικείμενο `HorizontalArrangement` και το τοποθετούμε στο κάτω μέρος της οθόνης. Στο πλαίσιο αυτό σέρνουμε δυο νέα αντικείμενα `Button`, ώστε να βρίσκονται οριζόντια το ένα δίπλα στο άλλο.



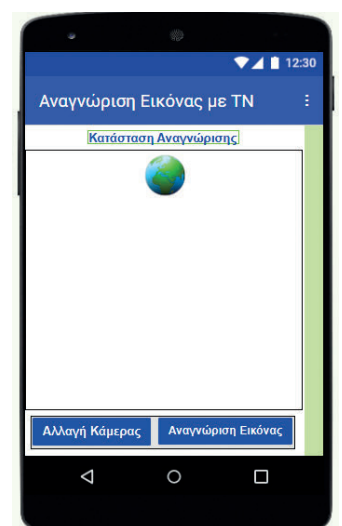
Στη συνέχεια πρέπει να συνδέσουμε το αντικείμενο `WebView` με το `Look`, έτσι ώστε η εικόνα που λαμβάνει η κάμερα να δίνεται ως είσοδος στο αντικείμενο `Look`, το οποίο με τη βοήθεια του εκπαιδευμένου νευρωνικού δικτύου θα προσπαθήσει να μαντέψει τι αντικείμενο βρίσκεται μπροστά στην κάμερα του κινητού μας. Στη διπλανή εικόνα επιλέγουμε το αντικείμενο `WebView1`, τύπου `WebView`.



Τέλος, μέσα από τις ιδιότητες των αντικειμένων `Button`, τα στοιχίζουμε στο κέντρο, τους αλλάζουμε χρώμα υποβάθρου και γραμματοσειράς και αυξάνουμε λίγο το μέγεθός τους. Προσθέτουμε το αντικείμενο `Status`, τύπου `Label`, στο πάνω μέρος της οθόνης, ώστε να βλέπουμε εκεί τα αποτελέσματα. Δίνουμε και κατάλληλα ονόματα στα αντικείμενα και στο κείμενο που εμφανίζεται στην οθόνη για κάθε αντικείμενο και έτσι ολοκληρώσαμε την διεπαφή μας, δηλαδή το μέρος της εφαρμογής μας που βλέπει ο χρήστης (front-end).

Στο αντικείμενο `status` έχουμε ορίσει να φαίνεται το κείμενο «Κατάσταση Αναγνώρισης», στο `button Toggle` το κείμενο «Αλλαγή Κάμερας» και στο `Classify` το κείμενο «Αναγνώριση Εικόνας».

Το επόμενο βήμα είναι να ασχοληθούμε με τη συμπεριφορά αυτών των αντικειμένων, όταν ενεργοποιηθούν, και τον κώδικα, δηλαδή τον αλγόριθμο που κρύβεται από πίσω, αυτό είναι το λεγόμενο back-end.



Ήρθε ή ώρα να μεταβούμε από την όψη Designer στην όψη Blocks.

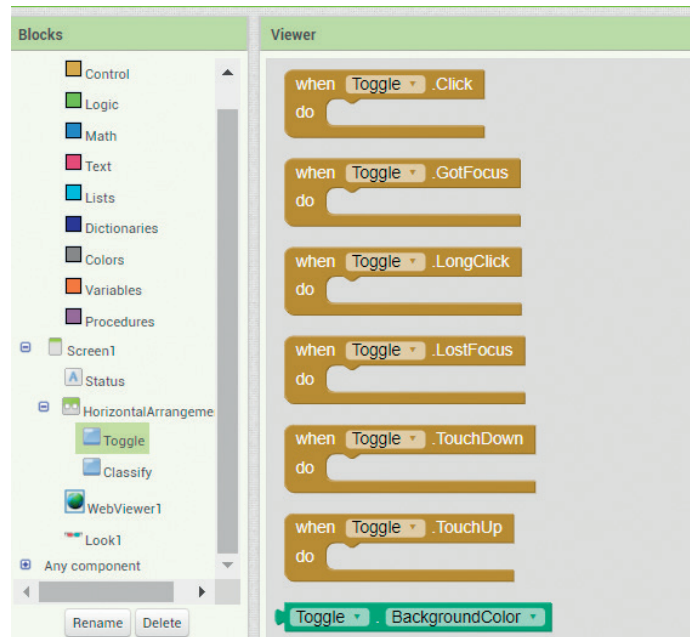
Συνδέοντας εντολές από τα αντικείμενα Button και το αντικείμενο Look1 κατασκευάζουμε τα παρακάτω τμήματα κώδικα:

Όταν πατηθεί το Toggle γίνεται εναλλαγή καμερών του κινητού,

```
when Toggle .Click
do call Look1 .ToggleCameraFacingMode
```

ενώ όταν πατηθεί το Classify η εφαρμογή μας προσπαθεί να αναγνωρίσει το αντικείμενο της εικόνας και να το κατατάξει σε κάποια από τις κατηγορίες που γνωρίζει.

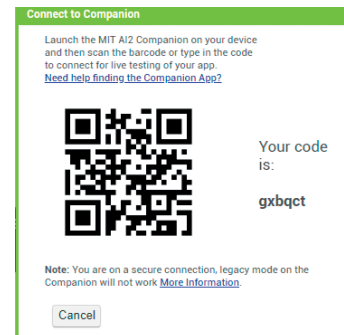
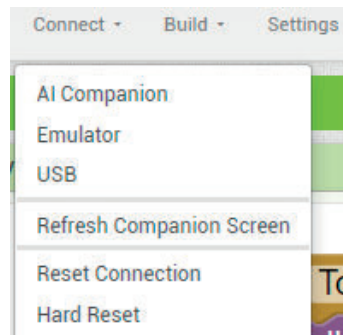
```
when Classify .Click
do call Look1 .ClassifyVideoData
```



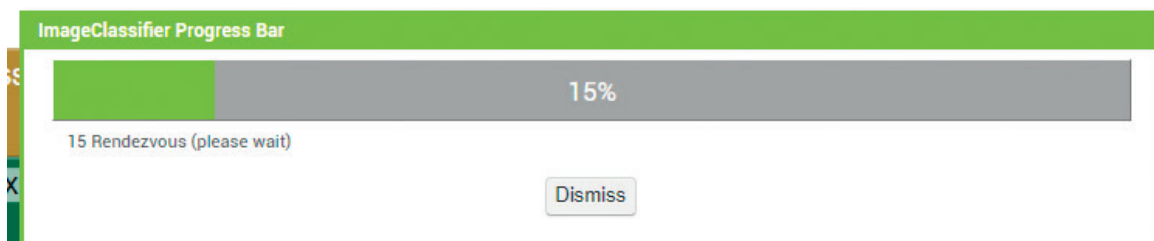
Στη συνέχεια το τελευταίο τμήμα κώδικα παίρνει το αποτέλεσμα του Look1 και το εμφανίζει ως κείμενο στη θέση του Status. Γι' αυτό θα χρειαστούμε ένα μπλοκ από το αντικείμενο Status και ένα από το result του Look1.

```
when Look1 .GotClassification
result
do set Status .Text to get result
```

Η εφαρμογή μας είναι έτοιμη. Μένει να τη συνδέσουμε με το κινητό μας. Ο πιο εύκολος τρόπος είναι, εφόσον κινητό ή ταμπλέτα είναι στο ίδιο δίκτυο με τον υπολογιστή μας, να επιλέξουμε AI Companion από το παρακάτω μενού και στη συνέχεια να σαρώσουμε το QR-Code που θα εμφανιστεί με το κινητό τηλέφωνο ή την ταμπλέτα.



Αναμένουμε να φορτώσει η εφαρμογή στο κινητό μας, ώστε να την εκτελέσουμε.



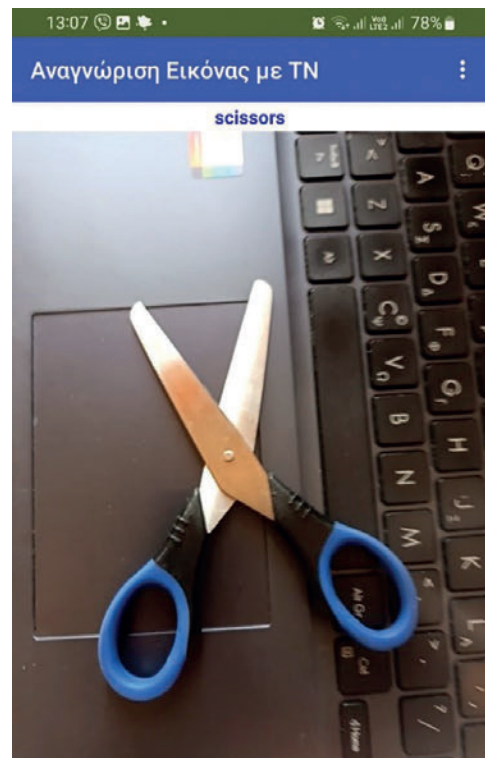
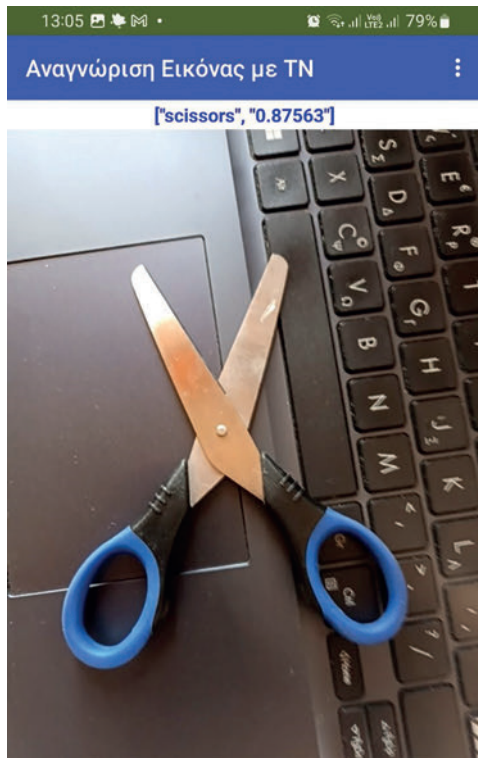
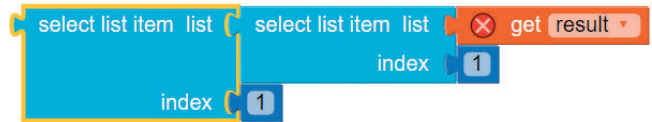
Παρακάτω φαίνονται μερικά παραδείγματα σάρωσης αντικειμένων και τα αποτελέσματά τους.



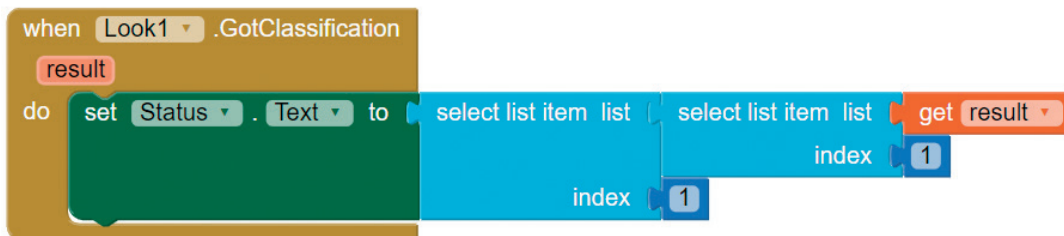
Για κάθε εικόνα η εφαρμογή μάς επιστρέφει μια λίστα με το όνομα του αντικειμένου και έναν αριθμό ο οποίος είναι ο συντελεστής βεβαιότητας. Στην τρίτη περίπτωση θεωρεί ότι η εικόνα που του δώσαμε είναι πληκτρολόγιο με βεβαιότητα 1 δηλαδή 100%, ενώ και στις άλλες δυο περιπτώσεις η βεβαιότητα είναι πολύ υψηλή. Μπορούμε, αν θέλουμε, να εμφανίζεται μόνο το πρώτο αντικείμενο της λίστας και από αυτό μόνο το όνομά του και όχι ο συντελεστής βεβαιότητας. Για να το πετύχουμε αυτό, θα πρέπει να εμφανίσουμε το πρώτο αντικείμενο της λίστας [["scissors", "0.95421"], ["hammer", "0.01518"]], αυτό που βρίσκεται στη θέση 1 και είναι πάλι μια λίστα η ["scissors", "0.95421"].

Γ' αυτό θα χρειαστούμε από την ομάδα Lists την εντολή select list item.

Αν θέλουμε πάλι το πρώτο αντικείμενο της λίστας ["scissors", "0.95421"], θα εφαρμόσουμε πάλι την ίδια εντολή στο αποτέλεσμα και θα πάρουμε μόνο το όνομα του πρώτου αντικειμένου.



Οπότε το τμήμα κώδικα θα πάρει την παρακάτω μορφή:



Δραστηριότητα 1

Μελετήστε το έργο PersonallImageClassifier στη διεύθυνση

<https://appinventor.mit.edu/explore/resources/ai/personal-image-classifier-part1>

Κατεβάστε την αντίστοιχη επέκταση και, αφού την εισάγετε στο έργο σας στο App Inventor, θα αναπτύξετε μια εφαρμογή στην οποία εσείς θα κατασκευάσετε και θα εκπαιδεύσετε το δικό σας νευρωνικό δίκτυο, ώστε να αναγνωρίζει συγκεκριμένες εικόνες.